TREE

🔹 1. What is a Tree?

A **Tree** is a hierarchical data structure made of **nodes** connected by **edges**.

👉 Unlike linear structures (array, linked list), trees are **non-linear**.

```
      A
     / \
    B   C
   / \
  D   E
```

🌿 Node

Each element in a tree is called a **node**.

👉 Example: A, B, C, D, E

🌿 Root

The **top-most node** of the tree.

👉 In above example: `A`

🌿 Leaf Node

A node with **no children**.

👉 `D`, `E`, `C`

🌿 Parent

A node that has children.

👉 `A` is parent of `B` and `C`

🌿 Child

Nodes that come under a parent.

👉 `B` and `C` are children of `A`

🌿 Internal Node

A node with at least one child.

👉 `A`, `B`

🌿 Path

Sequence of nodes from one node to another.

👉 Example: A → B → D

🌿 Edge

Connection between two nodes.

👉 Example: A → B

🌿 Degree of Node

Number of children a node has.

👉 Degree of B = 2

🌿 Degree of Tree

Maximum degree among all nodes.

👉 Here = 2

🌿 Depth of Node

Number of edges from root to that node.

👉 Depth of D = 2

🌿 Height of Node

Number of edges from node to deepest leaf.

👉 Height of A = 2

🌿 Height of Tree

Height of root node.

👉 Height = 2

🔹 4. Levels in Tree

Level starts from 0 (or sometimes 1 depending on definition).

Node	Level
A	0
B, C	1
D, E	2

🌳 **Binary Tree**

Each node has **at most 2 children**

🌳 **Full Binary Tree**

Every node has **0 or 2 children**

🌳 **Complete Binary Tree**

All levels filled except possibly last, filled left to right

🌳 **Perfect Binary Tree**

All nodes have 2 children and all leaves at same level

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Tree types

🔹 1. What is a Binary Tree?

A **Binary Tree** is a tree where each node has **at most 2 children**:

- Left child
- Right child

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

🔹 2. Maximum Nodes at a Level

👉 At level `L`, maximum nodes:

$$\text{Max Nodes} = 2^L$$

🔹 3. Maximum Nodes in a Tree

👉 If height = `h`:

$$\text{Max Nodes} = 2^{h+1} - 1$$

🔹 4. Minimum Nodes in a Binary Tree

👉 Minimum nodes for height `h`:

$$\text{Min Nodes} = h + 1$$

👉 (Skewed tree case)

🔸 Example:

Height = 3 → Minimum nodes = 4

5. Number of Edges

$$\text{Edges} = \text{Nodes} - 1$$

🔹 6. Height of Binary Tree

👉 For `n` nodes:

- **Minimum height (best case)**:

$$\log_2(n+1) - 1$$

- **Maximum height (worst case)**:

$$n - 1$$

🔹 7. Leaf Nodes Property

👉 Maximum number of leaf nodes:

$$\text{Leaf Nodes} = 2^h$$

🔹 8. Relation Between Nodes

In a **full binary tree**:

$$\text{Leaf Nodes} = \text{Internal Nodes} + 1$$

🔹 9. Complete Binary Tree Property

👉 For any node at index `i` (array representation):

- Left child → `2i + 1`
- Right child → `2i + 2`
- Parent → `(i - 1) / 2`

🔹 1. Perfect Binary Tree

✅ Definition:

A binary tree is **perfect** if:

- All internal nodes have **exactly 2 children**
- All **leaf nodes are at the same level**
- Tree is **completely full**

```
      A
     / \
    B   C
   / \ / \
  D  E F  G
```

🔹 2. Complete Binary Tree

✅ Definition:

A binary tree is **complete** if:

- All levels are filled **except possibly the last**
- Last level is filled **from left to right**

```
      A
     / \
    B   C
   / \ /
  D  E F
```

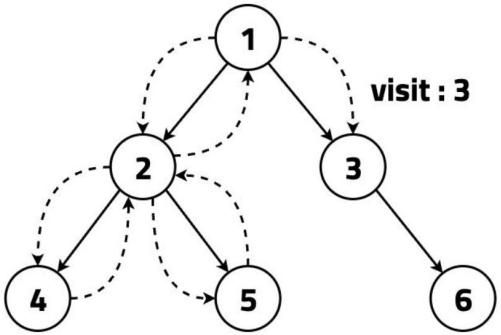
Tree Traversal

🌳 Tree Traversals

Definition:

Tree traversal is the process of **visiting each node of a tree exactly once** in a specific order.

1. Preorder Traversal (Root → Left → Right)



visit : 3

As we backtrack from 2 visit the right child of 1

preorder = [1, 2, 4, 5, 3]

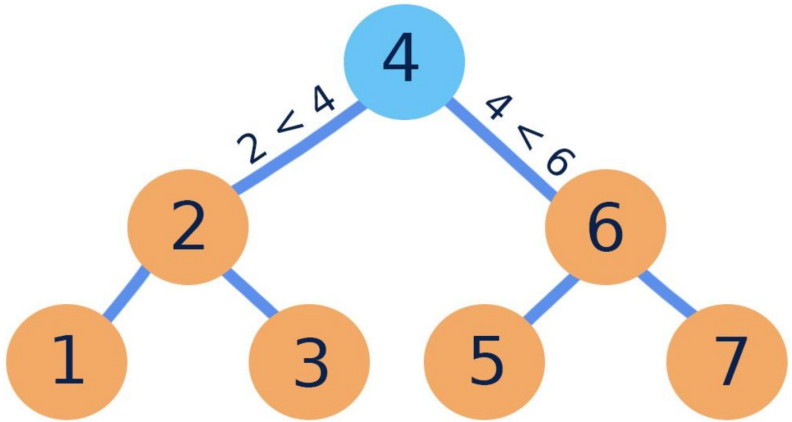
Steps:

1. Visit the root node
2. Traverse the left subtree
3. Traverse the right subtree

Use cases:

- Copying a tree
- Prefix expression (Polish notation)

2. Inorder Traversal (Left → Root → Right)



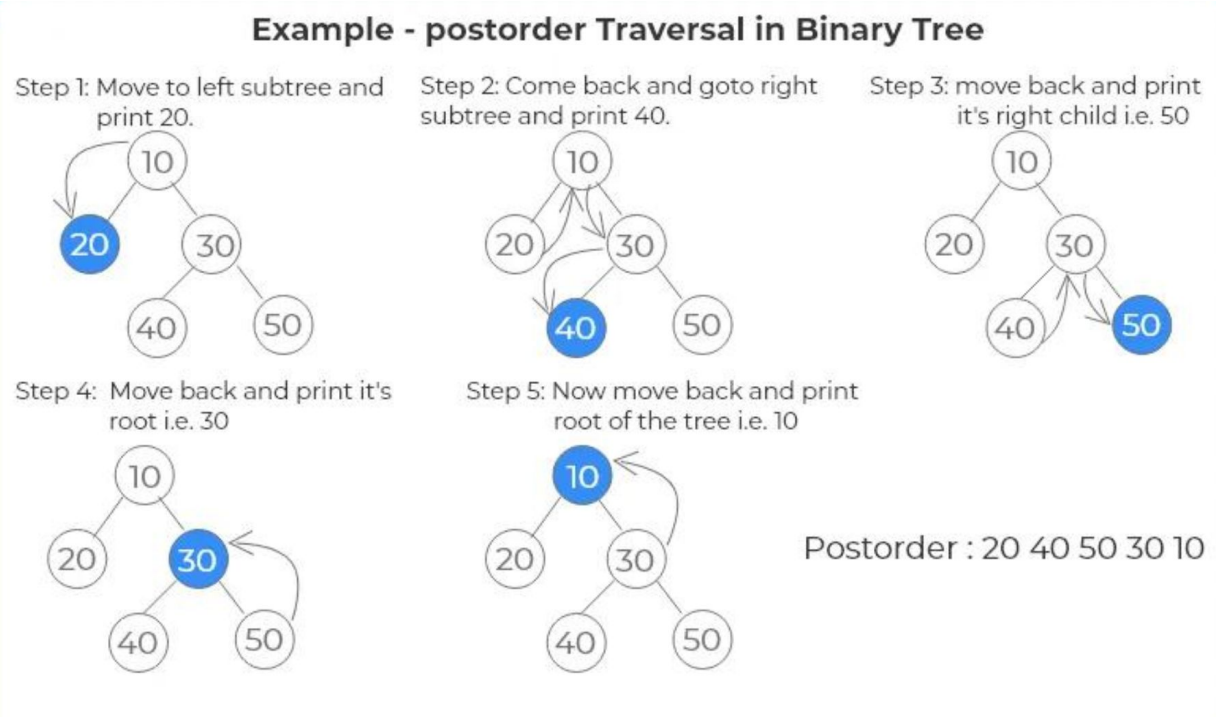
Steps:

1. Traverse left subtree
2. Visit root
3. Traverse right subtree

Use cases:

- Produces **sorted output** in Binary Search Tree (BST)

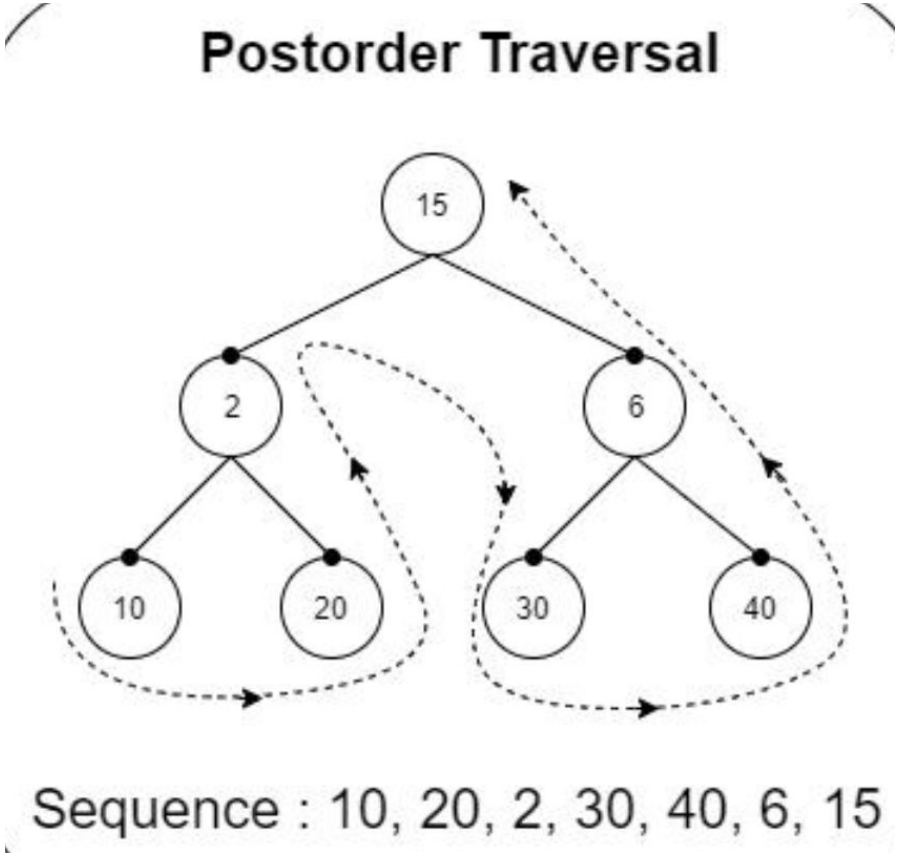
3. Postorder Traversal (Left → Right → Root)



Steps:

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

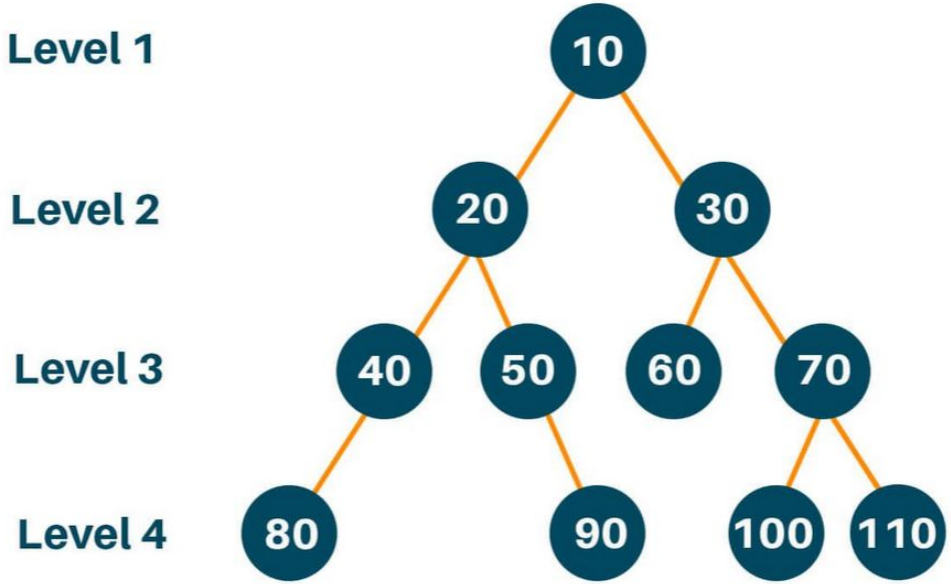
3. Postorder Traversal (Left → Right → Root)



Steps:

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

4. Level Order Traversal (Breadth-First)



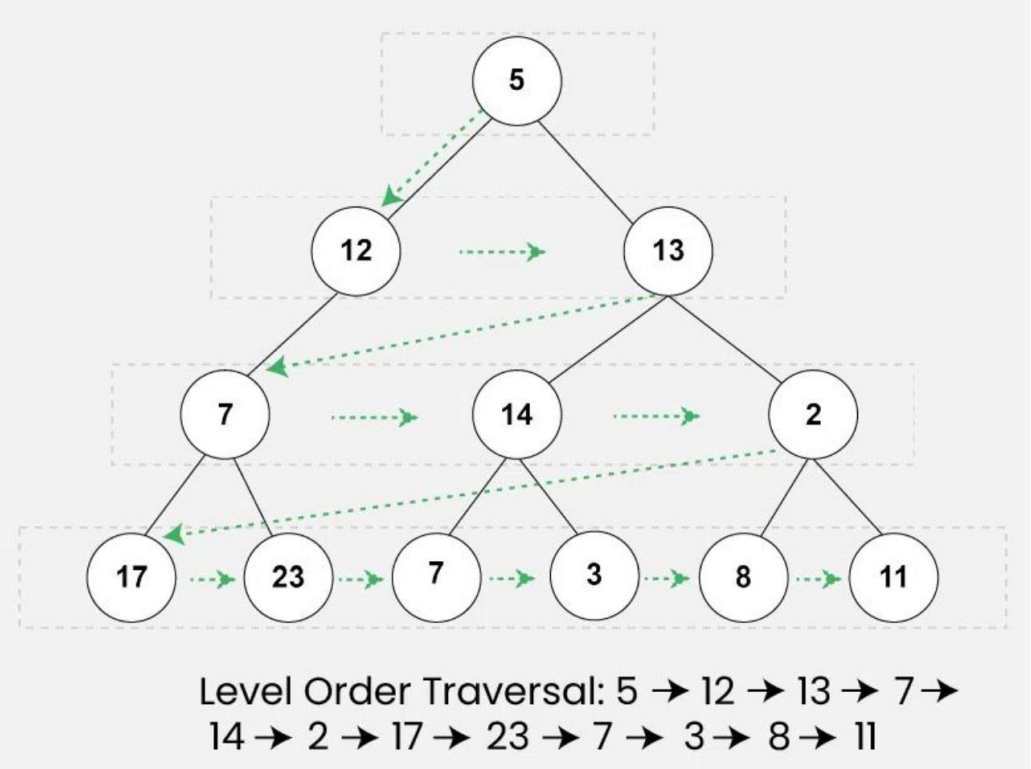
At level 1: [10]

At level 2: [20, 30]

At level 3: [40, 50, 60, 70]

At level 4: [80, 90, 100, 110]

4. Level Order Traversal (Breadth-First)



Steps:

- Visit nodes **level by level** from top to bottom
- Uses a **queue**

Use cases:

- Finding shortest path in trees
- BFS-based problems

InOrder(root) visits nodes in the following order:

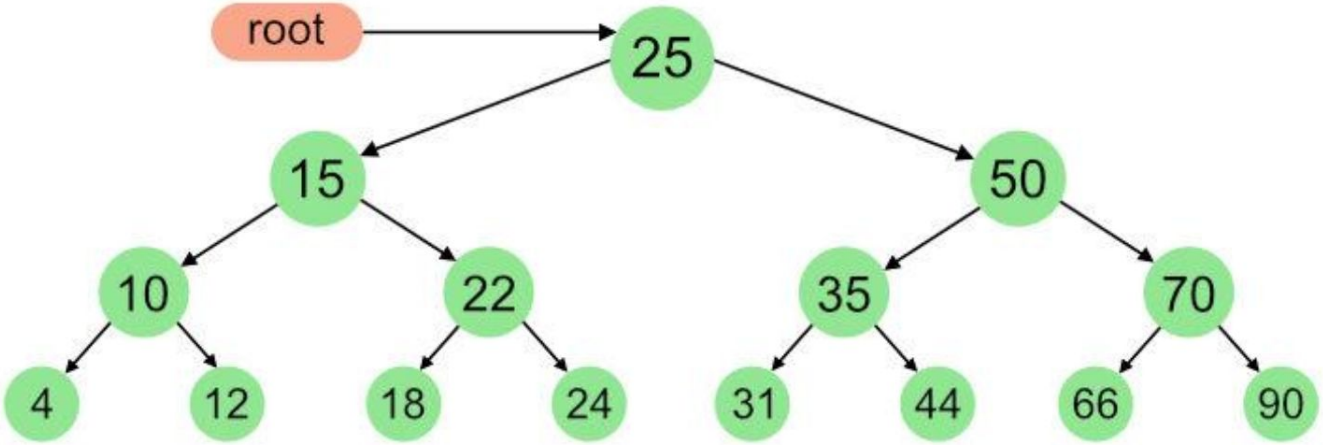
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

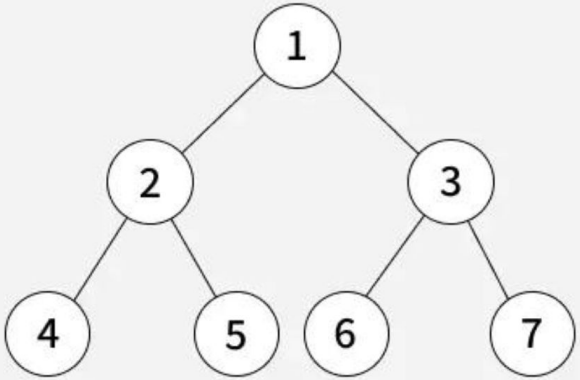
A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25





Inorder Traversal

4	2	5	1	6	3	7

Preorder Traversal

1	2	4	5	3	6	7

Postorder Traversal

4	5	2	6	7	3	1

Level order Traversal

1	2	3	4	5	6	7

💡 Key Points

- Preorder, Inorder, Postorder → **Depth First Search (DFS)**
- Level Order → **Breadth First Search (BFS)**
- Inorder gives **sorted output in BST**
- Postorder useful for **deletion and evaluation**

1. What is a Binary Search Tree (BST)?

A **Binary Search Tree (BST)** is a special type of binary tree in which each node follows a specific ordering rule:

- The left subtree contains values **less than** the node.
- The right subtree contains values **greater than** the node.
- Both left and right subtrees are also BSTs.

2.1 Ordering Property

For any node `N`:

- All elements in the left subtree < `N`
- All elements in the right subtree > `N`

2.2 No Duplicate Elements (usually)

- Standard BSTs do not allow duplicate values.
- (Some variations allow duplicates with specific rules.)

2.3 Recursive Structure

- Each subtree is itself a BST.
- This makes BSTs suitable for recursive algorithms.

2.4 Inorder Traversal Gives Sorted Output

- Performing **inorder traversal (Left → Root → Right)** returns elements in **ascending order**.

2.5 Time Complexity Dependence

- Depends on tree height:
  - Best/Average case: **O(log n)**
  - Worst case (skewed tree): **O(n)**

3.4 Traversals

1. Inorder (LNR)

- Left → Root → Right
- Output: **Sorted order**

2. Preorder (NLR)

- Root → Left → Right
- Useful for copying tree

3. Postorder (LRN)

- Left → Right → Root
- Useful for deletion

3.1 Insertion

Steps:

1. Start at the root.
2. Compare value with current node:
   - If smaller → go left
   - If larger → go right
3. Repeat until a null position is found.
4. Insert the node.

Time Complexity:

- Average: O(log n)
- Worst: O(n)

3.2 Searching

Steps:

1. Start at root.
2. Compare key with node:
   - If equal → found
   - If smaller → search left
   - If larger → search right

Time Complexity:

- Average: O(log n)
- Worst: O(n)

🌳 Initial BST (Common Example)

```
        50
       /  \
     30    70
    / \    / \
  20  40  60  80
```

🔴 CASE 1: Deleting a Leaf Node (No Children)

```
        50
       /  \
     30    70
    / \    / \
  [20] 40 60 80
```

After:

```
     50
    /  \
  30    70
   \    / \
   40  60  80
```

Action:

- Node **20 has no children**
- Simply **delete it**

Key Point:

Leaf node → directly remove, no restructuring needed

🟡 CASE 2: Deleting a Node with One Child

```
      50
     /
   30
     \
      40
```

👉 Delete: 30

```
      50
     /
   40
```

Action:

- Node **30 has one child (40)**
- Replace 30 with its child

Key Point:

One child → replace node with its child

🔵 CASE 3: Deleting a Node with Two Children

```
      [50]
     /    \
   30      70
  / \     / \
20  40   60  80
```

🔹 Step 1: Find Inorder Successor

👉 Smallest value in right subtree = **60**

🔹 Step 2: Replace

Replace **50** → **60**

```
      60
     /  \
   30    70
  / \     \
20  40     80
```

🔹 Step 3: Delete Duplicate (60)

Remove original **60** from right subtree:

Key Point:

Two children →
Replace with **inorder successor (min of right subtree)**
Then delete that successor

🌳 🔹 What is a Balanced BST?

👉 A **Balanced Binary Search Tree** is a BST where:

> The height of the tree is kept **minimum (≈ log n)** so that operations remain efficient.

🔹 Why Balance is Needed?

✖ Normal BST Problem:

If inserted in sorted order:

```
10 → 20 → 30 → 40
```

```
10
  \
   20
     \
      30
        \
         40
```

👉 This becomes like a **linked list**

⛔ Time Complexity → **O(n)**

✅ Balanced BST:

Keeps tree like:

```
      20
     /  \
   10    30
            \
             40
```

🔹 Types of Balanced BST (IMPORTANT)

1️⃣ AVL Tree

- Most commonly asked in exams
- Balance factor = **-1, 0, +1**
- Uses **rotations**

2️⃣ Red-Black Tree

- Used in real systems (like maps, sets)
- Less strict than AVL
- Uses **color properties**

3️⃣ B-Tree / B+ Tree

- Used in databases & file systems
- Multi-level balanced trees

A **Threaded Binary Tree** is a special type of binary tree designed to make **tree traversal faster and more efficient** by avoiding the use of stacks or recursion.

🌳 🔹 What is a Threaded Binary Tree?

👉 In a normal binary tree:

- Many **left and right pointers are NULL**
- These NULL pointers are wasted

👉 In a **Threaded Binary Tree**:

- These NULL pointers are replaced with **“threads”**
- Threads point to the **next node in traversal order** (usually inorder)

Normal Binary Tree

```
      10
    /   \
  5      15
```

Inorder Traversal:

```
5 → 10 → 15
```

Threaded Binary Tree Concept

- Node 5's right NULL → points to 10
- Node 15's left NULL → points to 10

👉 These links are called **threads**

🔹 Types of Threaded Binary Trees

1️⃣ Single Threaded

- Only one side is threaded
  - Either left OR right

2️⃣ Double Threaded

- Both left and right NULL pointers are used as threads

🔹 Types Based on Traversal

- **Inorder Threaded** (most common)
- Preorder Threaded
- Postorder Threaded

```
struct Node {
    int data;
    struct Node *left, *right;
    int lthread, rthread; // 1 = thread, 0 = child
};
```

🔹 Why Use Threaded Binary Tree?

✅ **Advantages:**

- No need for **recursion or stack**
- Faster traversal
- Better memory utilization (uses NULL pointers)

❌ Disadvantages:

- More complex implementation
- Extra flags needed to distinguish:
  - child pointer vs thread

3.3 Deletion

Deletion is the most complex operation. There are **3 cases**:

Case 1: Node is a Leaf

- Simply remove the node.

Case 2: Node has One Child

- Replace the node with its child.

Case 3: Node has Two Children

- Replace with:
  - **Inorder successor** (smallest in right subtree), OR
  - **Inorder predecessor** (largest in left subtree)
- Then delete that node.